

Reinforcement Learning

On-policy Prediction with Approximation: Part II

Min-hwan Oh

Seoul National University

TD with Function Approximation

- Recall (one-step) TD method with linear function approximation

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose $A \sim \pi(\cdot|S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$

$S \leftarrow S'$

 until S is terminal

Least Squares TD

- Direct computation of the TD fixed point instead of the iterative method
- Recall: TD(0) with linear function approximation converges asymptotically to the TD fixed point

$$\mathbf{w}_{TD} \doteq \mathbf{A}^{-1}\mathbf{b}$$

- ▶ $\mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t]$
- ▶ $\mathbf{A} \doteq \mathbb{E}[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T]$

- The Least Squares TD Algorithm (LSTD) computes directly estimates for $\mathbf{A}$ and $\mathbf{b}$:

$$\hat{\mathbf{b}}_t \doteq \sum_{k=0}^{t-1} R_{k+1}\mathbf{x}_k$$
$$\hat{\mathbf{A}}_t \doteq \sum_{k=0}^{t-1} \mathbf{x}_k(\mathbf{x}_k - \gamma\mathbf{x}_{k+1})^T + \epsilon\mathbf{I}$$

where $\mathbf{I}$ identity matrix and small $\epsilon > 0$

LSTD Algorithm

- LSTD estimates the fixed point as:

$$\mathbf{w}_t \doteq \hat{\mathbf{A}}_t^{-1} \hat{\mathbf{b}}_t$$

LSTD for estimating $\hat{v} = \mathbf{w}^\top \mathbf{x}(\cdot) \approx v_\pi$ ($O(d^2)$ version)

Input: feature representation $\mathbf{x} : \mathcal{S}^+ \rightarrow \mathbb{R}^d$ such that $\mathbf{x}(\text{terminal}) = \mathbf{0}$

Algorithm parameter: small $\varepsilon > 0$

$$\widehat{\mathbf{A}}^{-1} \leftarrow \varepsilon^{-1} \mathbf{I}$$

A $d \times d$ matrix

$$\hat{\mathbf{b}} \leftarrow \mathbf{0}$$

A d -dimensional vector

Loop for each episode:

Initialize S ; $\mathbf{x} \leftarrow \mathbf{x}(S)$

Loop for each step of episode:

Choose and take action $A \sim \pi(\cdot|S)$, observe R, S' ; $\mathbf{x}' \leftarrow \mathbf{x}(S')$

$$\mathbf{v} \leftarrow \widehat{\mathbf{A}}^{-1\top} (\mathbf{x} - \gamma \mathbf{x}')$$

$$\widehat{\mathbf{A}}^{-1} \leftarrow \widehat{\mathbf{A}}^{-1} - (\widehat{\mathbf{A}}^{-1} \mathbf{x}) \mathbf{v}^\top / (1 + \mathbf{v}^\top \mathbf{x})$$

$$\hat{\mathbf{b}} \leftarrow \hat{\mathbf{b}} + R \mathbf{x}$$

$$\mathbf{w} \leftarrow \widehat{\mathbf{A}}^{-1} \hat{\mathbf{b}}$$

$$S \leftarrow S'; \mathbf{x} \leftarrow \mathbf{x}'$$

until S' is terminal

Beyond Linear Function Approximation

- One can consider non-linear function approximation, e.g., neural networks
- Approximate $\hat{v}(s, \mathbf{w})$ with neural networks with weights $\mathbf{w}$
- But, we don't have global guarantees on convergence (yet)
- Why should we care about guarantees on linear cases?

Control with Function Approximation

- As usual, we now learn the **control** problem
- Introduce parametric approximation of the action-value function $\hat{q}(s, a, \mathbf{w}) \approx q_*(s, a)$ instead of $\hat{v}(s, \mathbf{w})$.
- e.g., in linear case, $\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^\top \mathbf{w}$
 - ▶ $\mathbf{x}(s, a) \in \mathbb{R}^p$ is a feature vector of state-action pair
 - ▶ e.g., stacked vector: $\mathbf{x}(s, a) = \begin{bmatrix} \dot{\mathbf{x}}(s) \\ \ddot{\mathbf{x}}(a) \end{bmatrix}$, where $\dot{\mathbf{x}}(s)$ is a state feature vector and $\ddot{\mathbf{x}}(a)$ is an action feature vector.
Vectorized outer product: $\mathbf{x}(s, a) = \text{vec}(\dot{\mathbf{x}}(s)\ddot{\mathbf{x}}(a)^\top)$, etc.
- Semi-gradient Sarsa: an extension of the semi-gradient TD(0) with $\hat{q}$.

Episodic Semi-gradient Control

- Approximate action-value function $\hat{q} \approx q_\pi$ is the parametrized function with weight vector $\mathbf{w}$.
- The general gradient-descent update for action-value prediction:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{q}(S_t, A_t, \mathbf{w}_t)] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

- U_t is the target:
 - ▶ e.g., $U_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$ for one-step Sarsa
 - ▶ can be the full MC return or any of the n -step Sarsa returns
- With one-step sarsa target, the method is called “episodic semi-gradient one-step Sarsa”

Episodic Semi-Gradient Sarsa

- Greedy action selection $A_t^* = \underset{a}{\operatorname{argmax}} \hat{q}(S_t, a, \mathbf{w}_t)$
- Policy improvement: ϵ -greedy

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 If S' is terminal:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

 Go to next episode

 Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

$S \leftarrow S'$

$A \leftarrow A'$

Semi-Gradient n -step Sarsa

- Use n -step return as the target U_t :

$$\mathbf{w}_{t+n} = \mathbf{w}_{t+n-1} + \alpha [G_{t:t+n} + \hat{q}(S_t, A_t, \mathbf{w}_{t+n-1})] \nabla \hat{q}(S_t, A_t, \mathbf{w}_{t+n-1})$$

Episodic semi-gradient n -step Sarsa for estimating $\hat{q} \approx q_*$ or q_π

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Input: a policy π (if estimating q_π)

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$, a positive integer n

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

All store and access operations (S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

 Select and store an action $A_0 \sim \pi(\cdot | S_0)$ or ε -greedy wrt $\hat{q}(S_0, \cdot, \mathbf{w})$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take action A_t

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then:

$T \leftarrow t + 1$

 else:

 Select and store $A_{t+1} \sim \pi(\cdot | S_{t+1})$ or ε -greedy wrt $\hat{q}(S_{t+1}, \cdot, \mathbf{w})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then $G \leftarrow G + \gamma^n \hat{q}(S_{\tau+n}, A_{\tau+n}, \mathbf{w})$ ($G_{\tau:\tau+n}$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G - \hat{q}(S_\tau, A_\tau, \mathbf{w})] \nabla \hat{q}(S_\tau, A_\tau, \mathbf{w})$

 Until $\tau = T - 1$

Average Rewards: a new problem setting for continuing tasks

Classical settings for formulating the goal in MDP:

- episodic setting
- discounted setting

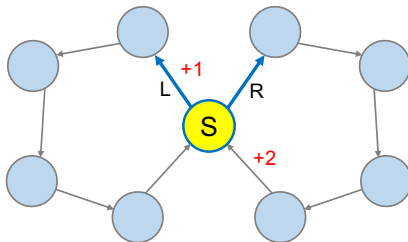
Average Rewards: a new problem setting for continuing tasks

Classical settings for formulating the goal in MDP:

- episodic setting
- discounted setting
- **average reward** setting

Why Average Reward?

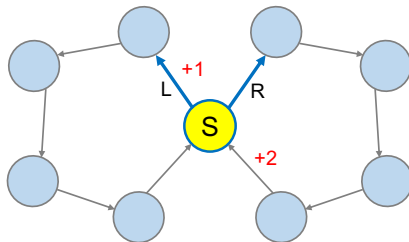
- Consider the following MDP with two actions in state S .¹



¹example from Marth White's Lecture

Why Average Reward?

- Consider the following MDP with two actions in state S .¹

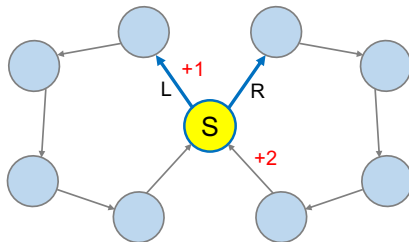


- Under discount γ , $q(S, L) = \frac{1}{1-\gamma^5}$ and $q(S, R) = \frac{2\gamma^4}{1-\gamma^5}$
 - with $\gamma = 0.5$, $q(S, L) \approx 1$ and $q(S, R) \approx 0.1 \Rightarrow$ Action L is preferred.
 - with $\gamma = 0.9$, $q(S, L) \approx 2.4$ and $q(S, R) \approx 3.2 \Rightarrow$ Action R is preferred.

¹example from Marth White's Lecture

Why Average Reward?

- Consider the following MDP with two actions in state S .¹



- Under discount γ , $q(S, L) = \frac{1}{1-\gamma^5}$ and $q(S, R) = \frac{2\gamma^4}{1-\gamma^5}$
 - with $\gamma = 0.5$, $q(S, L) \approx 1$ and $q(S, R) \approx 0.1 \Rightarrow$ Action L is preferred.
 - with $\gamma = 0.9$, $q(S, L) \approx 2.4$ and $q(S, R) \approx 3.2 \Rightarrow$ Action R is preferred.
- What happens if the length of the cycles become larger?

¹example from Marth White's Lecture

Average Rewards

- The quality of a policy π is defined as the average rate of reward $r(\pi)$

$$\begin{aligned} r(\pi) &= \lim_{h \rightarrow \infty} \frac{1}{h} \sum_{t=1}^h \mathbb{E}[R_t | A_{0:t-1} \sim \pi] \\ &= \lim_{t \rightarrow \infty} \mathbb{E}[R_t | A_{0:t-1} \sim \pi] \\ &= \sum_s \mu_\pi(s) \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) r \end{aligned}$$

- Valid under the ergodicity assumption: μ_π is the steady-state distribution and it exists for any π independently of S_0 .
 - ▶ steady state distribution is the distribution under which when you select an action you end up in the same distribution

$$\sum_s \mu_\pi(s) \sum_a \pi(a|s) p(s'|s, a) = \mu_\pi(s')$$

- ▶ In an ergodic MDP, starting state and early decisions have only a temporary effect; no effect in the limit.

Differential Return

- Differential return: returns defined in terms of differences between rewards and average rewards

$$G_t = R_{t+1} - r(\pi) + R_{t+1} - r(\pi) + \dots$$

- The corresponding value functions are differential value functions

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r - r(\pi) + v_{\pi}(s')]$$

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r|s, a) \left[r - r(\pi) + \sum_{a'} \pi(a'|s') q_{\pi}(s', a') \right]$$

$$v_*(s) = \max_a \sum_{s', r} p(s', r|s, a) \left[r - \max_{\pi} r(\pi) + v_*(s') \right]$$

$$q_*(s, a) = \sum_{s', r} p(s', r|s, a) \left[r - \max_{\pi} r(\pi) + \max_{a'} q_*(s', a') \right]$$

Differential TD Error

- Differential forms of the TD error:

$$\delta_t = R_{t+1} - \bar{R}_{t+1} + \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)$$

$$\delta_t = R_{t+1} - \bar{R}_{t+1} + \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t)$$

where $\bar{R}_t$ is an estimate at time t of the average reward $r(\pi)$.

- Average reward version of semi-gradient Sarsa is defined with the new version of the TD error:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \delta_t \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

Differential Sarsa Algorithm

- Parameter update:

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha \delta_t \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

where $\delta_t = R_{t+1} - \bar{R}_{t+1} + \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t)$

Differential semi-gradient Sarsa for estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step sizes $\alpha, \beta > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Initialize average reward estimate $\bar{R} \in \mathbb{R}$ arbitrarily (e.g., $\bar{R} = 0$)

Initialize state S , and action A

Loop for each step:

 Take action A , observe R, S'

 Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ϵ -greedy)

$\delta \leftarrow R - \bar{R} + \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})$

$\bar{R} \leftarrow \bar{R} + \beta \delta$

$\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \nabla \hat{q}(S, A, \mathbf{w})$

$S \leftarrow S'$

$A \leftarrow A'$

Differential n -Step Sarsa Algorithm

- Generalize to n -step bootstrapping

$$G_{t:t+n} = R_{t+1} - \bar{R}_{t+1} + \dots + R_{t+n} - \bar{R}_{t+n} + \hat{q}(S_{t+n}, A_{t+n}, \mathbf{w}_{t+n-1})$$

- The n -step TD error is then: $\delta_t = G_{t:t+n} - \hat{q}(S_t, A_t, \mathbf{w})$

Differential semi-gradient n -step Sarsa for estimating $\hat{q} \approx q_\pi$ or q_*

Input: a differentiable function $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$, a policy π

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Initialize average-reward estimate $\bar{R} \in \mathbb{R}$ arbitrarily (e.g., $\bar{R} = 0$)

Algorithm parameters: step size $\alpha, \beta > 0$, a positive integer n

All store and access operations (S_t , A_t , and R_t) can take their index mod $n + 1$

Initialize and store S_0 and A_0

Loop for each step, $t = 0, 1, 2, \dots$:

 Take action A_t

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 Select and store an action $A_{t+1} \sim \pi(\cdot | S_{t+1})$, or ε -greedy wrt $\hat{q}(S_{t+1}, \cdot, \mathbf{w})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

$$\delta \leftarrow \sum_{i=\tau+1}^{\tau+n} (R_i - \bar{R}) + \hat{q}(S_{\tau+n}, A_{\tau+n}, \mathbf{w}) - \hat{q}(S_\tau, A_\tau, \mathbf{w})$$

$$\bar{R} \leftarrow \bar{R} + \beta \delta$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \nabla \hat{q}(S_\tau, A_\tau, \mathbf{w})$$

Next class: Off-policy Methods with Approximation
(Chapter 11 of Sutton & Barto)